

最优控制课程论文

专 业： 控制科学与工程

学 生： 单文启

学 号： 22409105

轮腿式平衡机器人控制

陈阳, 王洪玺, 张兰勇

中文摘要: 针对轮腿平衡机器人的整体控制问题展开研究, 建立了机器人动力学模型, 采用 LQR (linear quadratic regulator) 算法对解耦后的平衡与纵向运动子系统进行分析, 并设计控制器。利用 VMC (virtual model control) 的思路, 将倒立摆机器人中的力矩转换成轮腿结构中关节力矩。通过搭建仿真平台 (Simulink Multibody) 对控制器的性能进行仿真实验。设计相应控制器对机器人高度与横滚姿态等状态进行控制, 并在实际机器人中进行验证整套控制器的性能, 具有一定的理论价值和实际应用价值。

关键词: 五连杆式轮腿结构; 平衡控制; 线性二次型调节器 (LQR); Webots; 虚拟模型控制 (VMC); 综合运动控制

中图分类号: TP24

文献标识码: A

Control of Wheel-legged Balancing Robot

CHEN Yang, WANG Hongxi, ZHANG Lanyong

Abstract:

This work studies the overall control problem of a multiterrain adaptive wheel-legged inverted pendulum robot. The dynamic model of the robot is also established. The linear quadratic regulator (LQR) algorithm analyzes the decoupled balance and longitudinal motion subsystems, and the controller is designed. Based on virtual model control, the torque in the inverted pendulum robot is converted into the joint torque in the wheel-leg structure. The controller's performance is simulated by building a simulation platform (Simulink Multibody). The corresponding controller is designed to control the robot's height and roll attitude. The performance of the whole controller is verified in the robot, which has certain theoretical and practical values.

Keywords : five-link wheel-legged structure; balance control; linear quadratic regulator (LQR); Webots; virtual model control (VMC); integrated motion control

1. 引言

如今, 工作环境的复杂性和多样化对移动机器人的机械结构设计提出了越来越高的要求。考虑到机器人在工作时需要快速响应和会遇到复杂路况这两大难题, 现有的技术中, 结合轮式和腿式移动机器人的运动优点的机器人得到了广泛的关注。

例如国外波士顿动力双足式人形机器人 Atlas, 四足式机器人 Spot 和两轮式人形机器人 Handle 以及瑞士苏黎世团队两轮式的 ANYmal 机器人 和 Ascento 机器人, 国内近年有腾讯 Robotics X 实验室发布的最新轮腿式机器人 Ollie 和本末科技公司的刑天机器人, 它们自身的拓扑结构各有千秋, 对于该类机器人的研究仍处于关键阶段。在 Ascento 机器人论文中, 实现了所谓的全身控制 (WBC), 这是一种现代的基于实时优化的控制方法, 需要机器人的完整模型。在多腿式机器人项目中, WBC 是一种性能良好的方法。在文中, 作者将五连杆机构引入轮腿机器人中, 并且应用基于输出调节和线性二次调节器 (LQR) 方法的线

性反馈控制器，效果显著，但同时系统的控制难度增大，使得系统的鲁棒性降低。而本文参考了文的拓扑结构，对机器人的动力学模型进行简化，在实现原本倒立摆机器人系统稳定性的同时提高系统的鲁棒性，并采取直觉控制中的虚拟模型控制（VMC）方法和线性二次调节器（LQR），将腿部结构的优点集于倒立摆机器人之上，实现整个轮腿机器人系统。

本次课程论文决定在该论文论文轮腿建模的基础上使用上海交通大学 RoboMaster 团队类 WBC 控制的方式完成整个仿真的复现。

2. 控制器设计

2.1 系统建模

对于机器人的平衡、纵向与转向运动控制，主要关注机器人上层机构与腿部姿态以及驱动轮的运动，并忽略机器人腿长变化，仅考虑腿的姿态，即驱动轮轴与腿部两关节电机转轴中心的连线相对于惯性系的角度。

2.1.1 模型定义

使用的模型定义与论文中不同，论文中使用单腿模型，并没有将旋转考虑在系统模型中，而本文模型决定采用上交团队的建模，也就是使用双腿倒立摆模型。得到如图 1 所示的 Webots 仿真模型和一侧倒立摆模型。

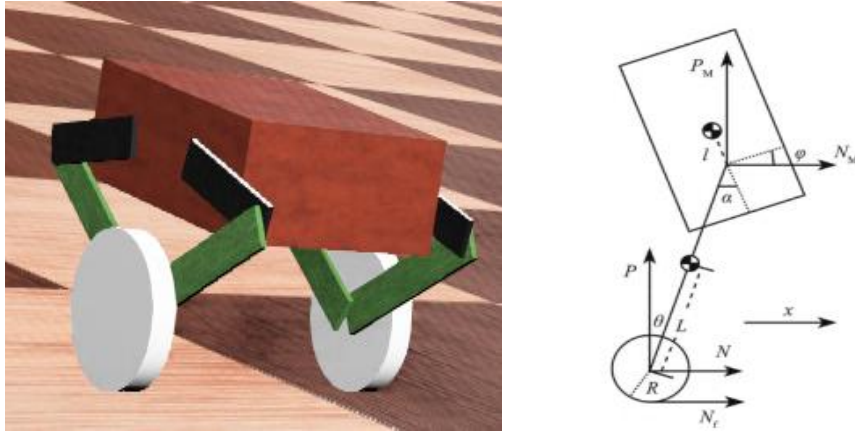


图 1 轮腿倒立摆模型

表 1 双腿模型轮腿倒立摆模型变量定义

符号	含义	单位
θ_b	机体倾斜角（自然坐标系法向）	rad
ϕ	偏航角, yaw	rad
s	自然坐标系下机器人水平方向移动距离	m
s_b	自然坐标系下机体质心水平方向移动距离	m
h_b	自然坐标系下机体质心竖直方向移动距离	m

$s_{l,i}$ ($i = l, r$)	自然坐标系下腿部质心水平方向移动距离	m
$h_{l,i}$ ($i = l, r$)	自然坐标系下腿部质心竖直方向移动距离	m
$T_{w,i}$ ($i = l, r$)	驱动轮转矩 (腿 — 轮)	$N \cdot m$
$T_{b,i}$ ($i = l, r$)	腿部转矩 (机体 — 腿)	$N \cdot m$
f_i ($i = l, r$)	地面对驱动轮摩擦力	N
$F_{wx,i}$ ($i = l, r$)	驱动轮对腿水平方向作用力	N
$F_{wh,i}$ ($i = l, r$)	驱动轮对腿竖直方向作用力	N
$F_{bx,i}$ ($i = l, r$)	腿对机体水平方向作用力	N
$F_{bh,i}$ ($i = l, r$)	腿对机体竖直方向作用力	N
$\theta_{w,i}$ ($i = l, r$)	驱动轮转角 (自然坐标系法向)	rad
$\theta_{l,i}$ ($i = l, r$)	腿倾斜角 (自然坐标系法向)	rad

表 2 轮腿倒立摆模型参数定义

符号	含义	单位
R_w	驱动轮半径	m
R_l	驱动轮轮距/2	m
l_i ($i = l, r$)	腿长	m
$l_{w,i}$ ($i = l, r$)	驱动轮到腿部质心距离	m
$l_{b,i}$ ($i = l, r$)	驱动轮到腿部质心距离	m
l_c	机体质心到腿部关节距离	m
m_w	驱动轮质量	kg
m_l	腿部质量	kg
m_b	机体质量	kg

I_w	驱动轮转动惯量（自然坐标系法向）	$kg \cdot m^2$
I_b	腿部转动惯量（自然坐标系法向）	$kg \cdot m^2$
I_b	机体转动惯量（自然坐标系法向）	$kg \cdot m^2$
I_z	机器人z轴转动惯量*2	$kg \cdot m^2$

2.1.3 运动学方程

根据变量、参数定义和几何关系得到系统的运动学方程，进而借得各变量得表达式

$$s = \frac{R_w}{2} (\theta_{w,l} + \theta_{w,r}) \quad (2.1)$$

$$h_b = \frac{l_l}{2} \cos \theta_{l,l} + \frac{l_r}{2} \cos \theta_{l,r} \quad (2.2)$$

$$s_{l,i} = R_w \theta_{w,i} + l_{w,i} \sin \theta_{l,i} \quad (2.3)$$

$$h_{l,i} = h_b - l_{b,i} \cos \theta_{l,i} \quad (2.4)$$

$$R_w \theta_{w,l} = s_b - R_l \phi - l_l \sin \theta_{l,l} \quad (2.5)$$

$$R_w \theta_{w,r} = s_b + R_l \phi - l_r \sin \theta_{l,r} \quad (2.6)$$

联立式 (2.5) (2.6)

$$\phi = \frac{R_w}{2R_l} (-\theta_{w,l} + \theta_{w,r}) - \frac{l_l}{2R_l} \sin \theta_{l,l} + \frac{l_r}{2R_l} \sin \theta_{l,r} \quad (2.7)$$

$$s_b = \frac{R_w}{2} (\theta_{w,l} + \theta_{w,r}) + \frac{l_l}{2} \sin \theta_{l,l} + \frac{l_r}{2} \sin \theta_{l,r} \quad (2.8)$$

式 (2.1) (2.2) (2.3) (2.4) (2.7) (2.8) 对时间分别求一/二阶导：

$$\dot{s} = \frac{R_w}{2} (\dot{\theta}_{w,l} + \dot{\theta}_{w,r}) \quad (2.9)$$

$$\ddot{s} = \frac{R_w}{2} (\ddot{\theta}_{w,l} + \ddot{\theta}_{w,r}) \quad (2.10)$$

$$\dot{\phi} = \frac{R_w}{2R_l} (-\dot{\theta}_{w,l} + \dot{\theta}_{w,r}) - \frac{l_l}{2R_l} \cos \theta_{l,l} \dot{\theta}_{l,l} + \frac{l_r}{2R_l} \cos \theta_{l,r} \dot{\theta}_{l,r} \quad (2.11)$$

$$\ddot{\phi} = \frac{R_w}{2R_l} (-\ddot{\theta}_{w,l} + \ddot{\theta}_{w,r}) - \frac{l_l}{2R_l} \cos \theta_{l,l} \ddot{\theta}_{l,l} + \frac{l_r}{2R_l} \cos \theta_{l,r} \ddot{\theta}_{l,r} + \frac{l_l}{2R_l} \sin \theta_{l,l} \dot{\theta}_{l,l}^2 - \frac{l_r}{2R_l} \sin \theta_{l,r} \dot{\theta}_{l,r}^2 \quad (2.12)$$

$$\dot{s}_b = \frac{R_w}{2} (\dot{\theta}_{w,l} + \dot{\theta}_{w,r}) + \frac{l_l}{2} \cos \theta_{l,l} \dot{\theta}_{l,l} + \frac{l_r}{2} \cos \theta_{l,r} \dot{\theta}_{l,r} \quad (2.13)$$

$$\ddot{s}_b = \frac{R_w}{2} (\ddot{\theta}_{w,l} + \ddot{\theta}_{w,r}) + \frac{l_l}{2} \cos \theta_{l,l} \ddot{\theta}_{l,l} + \frac{l_r}{2} \cos \theta_{l,r} \ddot{\theta}_{l,r} - \frac{l_l}{2} \sin \theta_{l,l} \dot{\theta}_{l,l}^2 - \frac{l_r}{2} \sin \theta_{l,r} \dot{\theta}_{l,r}^2 \quad (2.14)$$

$$\dot{h}_b = -\frac{l_l}{2}\sin\theta_{l,l}\dot{\theta}_{l,l} - \frac{l_r}{2}\sin\theta_{l,r}\dot{\theta}_{l,r} \quad (2.15)$$

$$\ddot{h}_b = -\frac{l_l}{2}\sin\theta_{l,l}\ddot{\theta}_{l,l} - \frac{l_r}{2}\sin\theta_{l,r}\ddot{\theta}_{l,r} - \frac{l_l}{2}\cos\theta_{l,l}\dot{\theta}_{l,l}^2 - \frac{l_r}{2}\cos\theta_{l,r}\dot{\theta}_{l,r}^2 \quad (2.16)$$

$$\dot{s}_{l,i} = R_w\dot{\theta}_{w,i} + l_{w,i}\cos\theta_{l,i}\dot{\theta}_{l,i} \quad (2.17)$$

$$\ddot{s}_{l,i} = R_w\ddot{\theta}_{w,i} + l_{w,i}\cos\theta_{l,i}\ddot{\theta}_{l,i} - l_{w,i}\sin\theta_{l,i}\dot{\theta}_{l,i}^2 \quad (2.18)$$

$$\dot{h}_{l,i} = \dot{h}_b + l_{b,i}\sin\theta_{l,i}\dot{\theta}_{l,i} \quad (2.19)$$

$$\ddot{h}_{l,i} = \ddot{h}_b + l_{b,i}\sin\theta_{l,i}\ddot{\theta}_{l,i} + l_{b,i}\cos\theta_{l,i}\dot{\theta}_{l,i}^2 \quad (2.20)$$

2.1.4 动力学方程

使用牛顿欧拉法建立系统的动力学方程，对于左右驱动轮

$$m_w R_w \ddot{\theta}_{w,i} = f_i - F_{wx,i} \quad (2.21)$$

$$I_w \ddot{\theta}_{w,i} = T_{lw,i} - f_i R_w \quad (2.22)$$

对于左右腿

$$m_l \ddot{s}_{l,i} = F_{wx,i} - F_{bx,i} \quad (2.23)$$

$$m_l \ddot{h}_{l,i} = F_{wh,i} - F_{bh,i} - m_l g \quad (2.24)$$

$$I_{l,i} \ddot{\theta}_{l,i} = (F_{wh,i} l_{w,i} + F_{bh,i} l_{b,i}) \sin\theta_{l,i} - (F_{wx,i} l_{w,i} + F_{bx,i} l_{b,i}) \cos\theta_{l,i} - T_{lw,i} + T_{bl,i} \quad (2.25)$$

对于机体

$$m_b \ddot{s}_b = F_{bs,l} + F_{bs,r} \quad (2.26)$$

$$m_b \ddot{h}_b = F_{bh,l} + F_{bh,r} - m_b g \quad (2.27)$$

$$I_b \ddot{\theta}_b = -(T_{bl,l} + T_{bl,r}) - (F_{bs,l} + F_{bs,r}) l_c \cos\theta_b + (F_{bh,l} + F_{bh,r}) l_c \sin\theta_b \quad (2.28)$$

机器人整体偏航方向旋转

$$I\ddot{\phi} = (-f_l + f_r) R_l \quad (2.29)$$

左右腿支持力相等

$$F_{wh,l} = F_{wh,r} \quad (2.30)$$

2.1.5 方程求解

将式 (2.1) ~ (2.4), (2.7) ~ (2.20) 带入式 (2.21) ~ (2.30), 分析动力学方程组, 共有 15 个未知量 (5 个广义坐标的二阶导数+10 个约束力), 15 个独立方程 (关于未知量线性), 方程有为一解。消去约束力, 并对腿部、机体倾角进行小角度近似, 得到如下方程组

$$\begin{aligned} & \left(I_w \frac{l_l}{R_w} + m_w R_w l_{b,l} \right) \ddot{\theta}_{w,l} + (m_l l_{w,l} l_{b,l} - I_l) \ddot{\theta}_{l,l} + \\ & \left(m_l l_{w,l} + \frac{1}{2} m_b l_l \right) g \theta_{l,l} + T_{b,l} - T_{lw,l} \left(1 + \frac{l_l}{R_w} \right) = 0 \quad (2.31) \end{aligned}$$

$$\begin{aligned} & \left(I_w \frac{l_r}{R_w} + m_w R_w l_{b,r} \right) \ddot{\theta}_{w,r} + (m_l l_{w,r} l_{b,r} - I_l) \ddot{\theta}_{l,r} + \\ & \left(m_l l_{w,r} + \frac{1}{2} m_b l_r \right) g \theta_{l,r} + T_{b,r} - T_{lw,r} \left(1 + \frac{l_r}{R_w} \right) = 0 \quad (2.32) \end{aligned}$$

$$\begin{aligned} & - \left(m_w R_w^2 + I_w + m_l R_w^2 + \frac{1}{2} m_b R_w^2 \right) \ddot{\theta}_{w,l} - \left(m_w R_w^2 + I_w + m_l R_w^2 + \frac{1}{2} m_b R_w^2 \right) \ddot{\theta}_{w,r} - \left(m_l R_w l_{w,l} + \frac{1}{2} m_b R_w l_l \right) \ddot{\theta}_{l,l} \\ & + \frac{1}{2} m_b R_w l_r \ddot{\theta}_{l,r} + T_{lw,l} + T_{lw,r} = 0 \quad (2.33) \end{aligned}$$

$$\begin{aligned} & \left(m_w R_w l_c + I_w \frac{l_c}{R_w} + m_l R_w l_c \right) \ddot{\theta}_{w,l} + \left(m_w R_w l_c + I_w \frac{l_c}{R_w} + m_l R_w l_c \right) \ddot{\theta}_{w,r} + m_l l_{w,l} l_c \ddot{\theta}_{l,l} + m_l l_{w,r} l_c \ddot{\theta}_{l,r} - I_l \ddot{\theta}_{l,r} + m_b g l_c \theta_b - \\ & (T_{lw,l} + T_{lw,r}) \frac{l_c}{R_w} - (T_{b,l} + T_{b,r}) = 0 \quad (2.34) \end{aligned}$$

$$\begin{aligned} & \frac{1}{2} I_z \frac{R_w}{R_l} + I_w \frac{R_l}{R_w} \ddot{\theta}_{w,l} - \frac{1}{2} I_z \frac{R_w}{R_l} \ddot{\theta}_{w,r} + I_w \frac{R_l}{R_w} \ddot{\theta}_{w,r} + \frac{1}{2} I_z \frac{l_l}{R_l} \ddot{\theta}_{l,l} - \\ & \frac{1}{2} I_z \frac{l_r}{R_l} \ddot{\theta}_{l,r} - T_{lw,l} \frac{R_l}{R_w} + T_{lw,r} \frac{R_l}{R_w} = 0 \quad (2.35) \end{aligned}$$

2.1.6 状态空间模型

之后步骤与论文中 1.1.3 节相同，定义非线性模型，定义状态向量和控制向量如下：

非线性模型： $\dot{x} = f(x, T, T_p)$

状态向量： $\mathbf{x} = [s \ \dot{s} \ \phi \ \dot{\phi} \ \theta_{l,l} \ \dot{\theta}_{l,l} \ \theta_{l,r} \ \dot{\theta}_{l,r} \ \theta_b \ \dot{\theta}_b]^T$

控制向量： $\mathbf{u} = [T_{lw,l} \ T_{lw,r} \ T_{bl,l} \ T_{bl,r}]^T$

根据式状态向量 $\mathbf{x}$ 和控制向量 $\mathbf{u}$ ，求非线性模型平衡点处的雅各比矩阵并对其进行线性化即为：

$$A = \frac{\partial f}{\partial x}(x, u), \quad B = \frac{\partial f}{\partial u}(x, u)$$

在非线性模型求解的基础上，再求解平衡点出的雅各比矩阵，有：

$$\begin{cases} \dot{\mathbf{x}} = A \mathbf{x} + B \mathbf{u} \\ \mathbf{y} = C \mathbf{x} \end{cases}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & a_{2,5} & 0 & a_{2,7} & 0 & a_{2,9} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & a_{4,5} & 0 & a_{4,7} & 0 & a_{4,9} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & a_{6,5} & 0 & a_{6,7} & 0 & a_{6,9} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & a_{8,5} & 0 & a_{8,7} & 0 & a_{8,9} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & a_{10,5} & 0 & a_{10,7} & 0 & a_{10,9} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ b_{2,1} & b_{2,2} & b_{2,3} & b_{2,4} \\ 0 & 0 & 0 & 0 \\ b_{4,1} & b_{4,2} & b_{4,3} & b_{4,4} \\ 0 & 0 & 0 & 0 \\ b_{6,1} & b_{6,2} & b_{6,3} & b_{6,4} \\ 0 & 0 & 0 & 0 \\ b_{8,1} & b_{8,2} & b_{8,3} & b_{8,4} \\ 0 & 0 & 0 & 0 \\ b_{10,1} & b_{10,2} & b_{10,3} & b_{10,4} \end{bmatrix}, \quad C = I_{10}$$

$$a_{i,j} = \begin{cases} \frac{R_w}{2} \left(\frac{\partial \ddot{\theta}_{w,l}}{\partial x_j} + \frac{\partial \ddot{\theta}_{w,r}}{\partial x_j} \right) & (i=2) \\ \frac{R_w}{2R_l} \left(-\frac{\partial \ddot{\theta}_{w,l}}{\partial x_j} + \frac{\partial \ddot{\theta}_{w,r}}{\partial x_j} \right) - \frac{l_l}{2R_l} \frac{\partial \ddot{\theta}_{l,l}}{\partial x_j} + \frac{l_r}{2R_l} \frac{\partial \ddot{\theta}_{l,r}}{\partial x_j} & (i=4) \\ \frac{\partial \dot{x}_i}{\partial x_j} & (i=6, 8, 10) \end{cases}$$

$$b_{i,j} = \begin{cases} \frac{R_w}{2} \left(\frac{\partial \ddot{\theta}_{w,l}}{\partial u_j} + \frac{\partial \ddot{\theta}_{w,r}}{\partial u_j} \right) & (i=2) \\ \frac{R_w}{2R_l} \left(-\frac{\partial \ddot{\theta}_{w,l}}{\partial u_j} + \frac{\partial \ddot{\theta}_{w,r}}{\partial u_j} \right) - \frac{l_l}{2R_l} \frac{\partial \ddot{\theta}_{l,l}}{\partial u_j} + \frac{l_r}{2R_l} \frac{\partial \ddot{\theta}_{l,r}}{\partial u_j} & (i=4) \\ \frac{\partial \dot{x}_i}{\partial u_j} & (i=6, 8, 10) \end{cases}$$

附 matlab 求解 A、B 矩阵代码如下：

```
clear all
clc
% 定义机器人机体参数
syms R_w % 驱动轮半径
syms R_l % 驱动轮轮距/2
syms l_l l_r % 左右腿长
syms l_wl l_wr % 驱动轮质心到左右腿部质心距离
syms l_bl l_br % 机体质心到左右腿部质心距离
syms l_c % 机体质心到腿部关节中心点距离
syms m_w m_l m_b % 驱动轮质量 腿部质量 机体质量
syms I_w % 驱动轮转动惯量 (自然坐标系法向)
syms I_ll I_lr % 驱动轮左右腿部转动惯量 (自然坐标系法向, 实际上会变化)
syms I_b % 机体转动惯量 (自然坐标系法向)
syms I_z % 机器人 z 轴转动惯量 (简化为常量)
% 定义其他独立变量并补充其导数
syms theta_wl theta_wr % 左右驱动轮转角
syms dtheta_wl dtheta_wr
syms ddtheta_wl ddtheta_wr ddtheta_ll ddtheta_lr ddtheta_b
```



```

% 定义状态向量
syms s ds phi dphi theta_ll dtheta_ll theta_lr dtheta_lr theta_b dtheta_b
% 定义控制向量
syms T_wl T_wr T_bl T_br
% 输入物理参数：重力加速度
syms g
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%Step 1: 解方程，求控制矩阵 A, B%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 通过方程组(2.31)-(2.35)，求出
ddtheta_wl,ddtheta_wr,ddtheta_ll,ddtheta_lr,ddtheta_b 表达式
eqn1 =
(I_w*I_l/R_w+m_w*R_w*I_l+m_l*R_w*I_bl)*ddtheta_wl+(m_l*I_w*I_bl-I_ll)*ddtheta_ll+(m_l*I_w+m_b*I_l/2)*g*theta_ll+T_bl-T_wl*(1+I_l/R_w)==0;
eqn2 =
(I_w*I_r/R_w+m_w*R_w*I_r+m_l*R_w*I_br)*ddtheta_wr+(m_l*I_w*I_br-I_lr)*ddtheta_lr+(m_l*I_w+m_b*I_r/2)*g*theta_lr+T_br-T_wr*(1+I_r/R_w)==0;
eqn3 =
-(m_w*R_w*R_w+I_w+m_l*R_w*R_w+m_b*R_w*R_w/2)*ddtheta_wl-(m_w*R_w*R_w+I_w+m_l*R_w*R_w+m_b*R_w*R_w/2)*ddtheta_wr-(m_l*R_w*I_w+m_b*R_w*I_l/2)*ddtheta_ll-(m_l*R_w*I_w+m_b*R_w*I_r/2)*ddtheta_lr+T_wl+T_wr==0;
eqn4 =
(m_w*R_w*I_c+I_w*I_c/R_w+m_l*R_w*I_c)*ddtheta_wl+(m_w*R_w*I_c+I_w*I_c/R_w+m_l*R_w*I_c)*ddtheta_wr+m_l*I_w*I_c*ddtheta_ll+m_l*I_w*I_c*ddtheta_lr-I_b*ddtheta_b+m_b*g*I_c*theta_b-(T_wl+T_wr)*I_c/R_w-(T_bl+T_br)==0;
eqn5 =
((I_z*R_w)/(2*R_l)+I_w*R_l/R_w)*ddtheta_wl-((I_z*R_w)/(2*R_l)+I_w*R_l/R_w)*ddtheta_wr+(I_z*I_l)/(2*R_l)*ddtheta_ll-(I_z*I_r)/(2*R_l)*ddtheta_lr-T_wl*R_l/R_w+T_wr*R_l/R_w==0;
[ddtheta_wl,ddtheta_wr,ddtheta_ll,ddtheta_lr,ddtheta_b] =
solve(eqn1,eqn2,eqn3,eqn4,eqn5,ddtheta_wl,ddtheta_wr,ddtheta_ll,ddtheta_lr,ddtheta_b);
% 通过计算雅可比矩阵的方法得出控制矩阵 A, B 所需要的全部偏导数
J_A =
jacobian([ddtheta_wl,ddtheta_wr,ddtheta_ll,ddtheta_lr,ddtheta_b],[theta_ll,theta_lr,theta_b]);
J_B =
jacobian([ddtheta_wl,ddtheta_wr,ddtheta_ll,ddtheta_lr,ddtheta_b],[T_wl,T_wr,T_bl,T_br]);
% 定义矩阵 A, B, 将指定位置的数值根据上述偏导数计算出来并填入
A = sym('A',[10 10]);
B = sym('B',[10 4]);
% 填入 A 数据: a25,a27,a29,a45,a47,a49,a65,a67,a69,a85,a87,a89,a105,a107,a109
for p = 5:2:9
    A_index = (p - 3)/2;
    A(2,p) = R_w*(J_A(1,A_index) + J_A(2,A_index))/2;

```

```

    A(4,p) = (R_w*(- J_A(1,A_index) + J_A(2,A_index)))/(2*R_1) -
(1_l1*J_A(3,A_index))/(2*R_1) + (1_r*J_A(4,A_index))/(2*R_1);
    for q = 6:2:10
        A(q,p) = J_A(q/2,A_index);
    end
end
% A 的以下数值为 1: a12,a34,a56,a78,a910, 其余数值为 0
for r = 1:10
    if rem(r,2) == 0
        A(r,1) = 0; A(r,2) = 0; A(r,3) = 0; A(r,4) = 0; A(r,6) = 0; A(r,8) = 0;
        A(r,10) = 0;
    else
        A(r,:) = zeros(1,10);
        A(r,r+1) = 1;
    end
end
% 填入 B 数据:
b21,b22,b23,b24,b41,b42,b43,b44,b61,b62,b63,b64,b81,b82,b83,b84,b101,b102,b
103,b104,
for h = 1:4
    B(2,h) = R_w*(J_B(1,h) + J_B(2,h))/2;
    B(4,h) = (R_w*(- J_B(1,h) + J_B(2,h)))/(2*R_1) - (1_l1*J_B(3,h))/(2*R_1) +
(1_r*J_B(4,h))/(2*R_1);
    for f = 6:2:10
        B(f,h) = J_B(f/2,h);
    end
end
% B 的其余数值为 0
for e = 1:2:9
    B(e,:) = zeros(1,4);
end

```

至此本章状态空间模型的求解结束，运行代码即可得到具体的状态空间方程，篇幅原因这里不展示结果了。

2.1.7 控制器设计 LQR

机器人控制解耦为运动控制（Locomotion Controller）和腿部控制（Leg Controller）两部分。运动控制器通过调节驱动轮转矩和腿部关节转矩使机器人保持平衡和控制机器人移动、旋转；腿部控制器通过调节腿部支持力控制腿长，使机器人保持滚转（roll）方向稳定，并起到主动悬挂的作用。

根据状态空间模型实际状态反馈控制器 $\mathbf{u} = K_{4 \times 10}(\mathbf{x}_d - \mathbf{x})$

状态反馈矩阵 K 通过 Linear Quadratic Regulator (LQR) 计算得到，代价函数

$$J = \int_0^{\infty} (x' Q x + u' R u) dx$$

根据 LQR 可以得到状态反馈矩阵 K

$$K = R^{-1} B^T P$$

其中 P 满足 Riccati 方程

$$A^T P + PA + PBR^{-1}B^T P + Q = 0$$

由于轮腿式机器人腿部是一个相当重要的自由度,腿长实时变化但又是状态空间模型中的重要参数。LQR 控制器可以视为一个无限时域的无约束线性 MPC,所以系统模型不变,可以通过 matlab 的 LQR 函数离线求解 Riccati 方程,得到固定的反馈增益,只要系统模型权重不发生变化, K 就可以永久使用无需在线更新,这样的话可以在 Matlab 中提前求解出不同腿长的反馈增益 K 矩阵,在进行多项式拟合,就可以极大的减轻实际主控的压力。无需像 MPC 一样在线计算,所以 LQR 算法是可以在 STM32 这类主控上跑的。

A,B 是关于腿长的函数,为了降低运算量,在工作空间内采用多项式拟合的方式得到状态反馈矩阵 K

$$K(l_p, l_r) = K_{K,1} + K_{K,2}l_r + K_{K,3}l_r^2 + K_{K,4}l_p + K_{K,5}l_r^2 + K_{K,6}l_p l_r$$

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%Step 2: 输入参数（可以修改的部分）%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
g_ac = 9.81;
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%机器人机体与轮部参数%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
R_w_ac = 0.9; % 驱动轮半径 (单位: m)
R_l_ac = 0.25; % 两个驱动轮之间距离/2 (单位: m)
l_c_ac = 0.037; % 机体质心到腿部关节中心点距离 (单位: m)
m_w_ac = 0.8; m_l_ac = 1.6183599; m_b_ac = 11.542; % 驱动轮质量 腿部质量 机体质量 (单位: kg)
I_w_ac = (3510000)*10^(-7); % 驱动轮转动惯量 (单位: kg m^2)
I_b_ac = 0.260; % 机体转动惯量(自然坐标系法向) (单位: kg m^2)
I_z_ac = 0.226; % 机器人 z 轴转动惯量 (单位: kg m^2)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%机器人腿部参数（定腿长调试用）%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 如果需要使用此部分,请去掉 120-127 行以及 215-218 行注释,然后将 224 行之后的所有代码注释掉
% 或者点击左侧数字"224"让程序只运行行之前的内容并停止
l_l_ac = 0.18; % 左腿摆杆长度 (左腿对应数据) (单位: m)
l_wl_ac = 0.05; % 左驱动轮质心到左腿摆杆质心距离 (单位: m)
l_bl_ac = 0.13; % 机体转轴到左腿摆杆质心距离 (单位: m)
I_ll_ac = 0.02054500; % 左腿摆杆转动惯量 (单位: kg m^2)
l_r_ac = 0.18; % 右腿摆杆长度 (右腿对应数据) (单位: m)
    
```

```

l_wr_ac = 0.05;          % 右驱动轮质心到右腿摆杆质心距离          (单位:
m)
l_br_ac = 0.13;          % 机体转轴到右腿摆杆质心距离          (单位:
m)
I_lr_ac = 0.02054500;    % 右腿摆杆转动惯量
(单位: kg m^2)
% 机体转轴定义参考哈工程开源 (https://zhuanlan.zhihu.com/p/563048952), 是左右
% 两侧两个关节电机之间的中间点相连所形成的轴
% (如果目的是小板凳, 考虑使左右腿相关数据一致)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 根据不同腿长长度, 先针对左腿测量出对应的 l_wl, l_bl, 和 I_ll
% 通过以下方式记录数据: 矩阵分 4 列,
% 第一列为左腿腿长范围区间中所有小数点精度 0.01 的长度, 例如: 0.09, 0.18, 单位: m
% 第二列为 l_wl, 单位: m
% 第三列为 l_bl, 单位: m
% 第四列为 I_ll, 单位: kg m^2
% (注意单位别搞错!)
% 行数根据 L_0 范围区间 (, 单位 cm 时) 的整数数量进行调整
Leg_data_l = [0.11, 0.0480, 0.0620, 0.01819599;
              0.12, 0.0470, 0.0730, 0.01862845;
              0.13, 0.0476, 0.0824, 0.01898641;
              0.14, 0.0480, 0.0920, 0.01931342;
              0.15, 0.0490, 0.1010, 0.01962521;
              0.16, 0.0500, 0.1100, 0.01993092;
              0.17, 0.0510, 0.1190, 0.02023626;
              0.18, 0.0525, 0.1275, 0.02054500;
              0.19, 0.0539, 0.1361, 0.02085969;
              0.20, 0.0554, 0.1446, 0.02118212;
              0.21, 0.0570, 0.1530, 0.02151357;
              0.22, 0.0586, 0.1614, 0.02185496;
              0.23, 0.0600, 0.1700, 0.02220695;
              0.24, 0.0621, 0.1779, 0.02256999;
              0.25, 0.0639, 0.1861, 0.02294442;
              0.26, 0.0657, 0.1943, 0.02333041;
              0.27, 0.0676, 0.2024, 0.02372806;
              0.28, 0.0700, 0.2100, 0.02413735;
              0.29, 0.0713, 0.2187, 0.02455817;
              0.30, 0.0733, 0.2267, 0.02499030];
% 以上数据应通过实际测量或 sw 图纸获得
% 由于左右腿部数据通常完全相同, 我们通过复制的方式直接定义右腿的全部数据集
% 矩阵分 4 列, 第一列为右腿腿长范围区间中 (, 单位 cm 时) 的整数腿长 l_r*0.01, 第二列
% 为 l_wr, 第三列为 l_br, 第四列为 I_lr)
Leg_data_r = Leg_data_l;

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
%                                此处可以输入 QR 矩阵                                %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 矩阵 Q 中，以下列分别对应：
%      s      ds      phi      dphi      theta_ll dtheta_ll theta_lr dtheta_lr
% theta_b dtheta_b
lqr_Q = [1,      0,      0,      0,      0,      0,      0,      0,      0,
0;
          0,      2,      0,      0,      0,      0,      0,      0,      0,
0;
          0,      0,      12000,      0,      0,      0,      0,      0,
0,      0;
          0,      0,      0,      200,      0,      0,      0,      0,      0,
0;
          0,      0,      0,      0,      1000,      0,      0,      0,      0,
0;
          0,      0,      0,      0,      0,      1,      0,      0,      0,
0;
          0,      0,      0,      0,      0,      0,      1000,      0,      0,
0;
          0,      0,      0,      0,      0,      0,      0,      1,      0,
0;
          0,      0,      0,      0,      0,      0,      0,      0,
20000,      0;
          0,      0,      0,      0,      0,      0,      0,      0,      0,
1];
% 其中：
% s      : 自然坐标系下机器人水平方向移动距离，单位：m，ds 为其导数
% phi    : 机器人水平方向移动时 yaw 偏航角度，dphi 为其导数
% theta_ll: 左腿摆杆与竖直方向（自然坐标系 z 轴）夹角，dtheta_ll 为其导数
% theta_lr: 右腿摆杆与竖直方向（自然坐标系 z 轴）夹角，dtheta_lr 为其导数
% theta_b : 机体与自然坐标系水平夹角，dtheta_b 为其导数

% 矩阵中，以下列分别对应：
%      T_wl      T_wr      T_bl      T_br
lqr_R = [0.25,      0,      0,      0;
          0,      0.25,      0,      0;
          0,      0,      1.5,      0;
          0,      0,      0,      1.5];
% 其中：
% T_wl: 左侧驱动轮输出力矩
% T_wr: 右侧驱动轮输出力矩
% T_bl: 左侧髋关节输出力矩

```

```
% T_br: 右腿髋关节输出力矩
% 单位皆为 Nm
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%Step 2.5: 求解矩阵（定腿长调试用）%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 如果需要使用此部分，请去掉 120-127 行以及 215-218 行注释，然后将 224 行之后的所有代码注释掉，
% 或者点击左侧数字"224"让程序只运行行之前的内容并停止
K =
get_K_from_LQR(R_w,R_l,l_l,l_r,l_wl,l_wr,l_bl,l_br,l_c,m_w,m_l,m_b,I_w,I_ll
,I_lr,I_b,I_z,g, ...
    R_w_ac,R_l_ac,l_l_ac,l_r_ac,l_wl_ac,l_wr_ac,l_bl_ac,l_br_ac, ...

l_c_ac,m_w_ac,m_l_ac,m_b_ac,I_w_ac,I_ll_ac,I_lr_ac,I_b_ac,I_z_ac,g_ac, ...
    A,B,lqr_Q,lqr_R)
K = sprintf([strjoin(repmat({'%.5g'},1,size(K,2)),', ') '\n'], K.')
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%Step 3: 拟合控制律函数%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
sample_size = size(Leg_data_l,1)^2; % 单个 K_ij 拟合所需要的样本数
length = size(Leg_data_l,1); % 测量腿部数据集的行数
% 定义所有 K_ij 依据 l_l,l_r 变化的表格，每一个表格有 3 列，第一列是 l_l，第二列
% 是 l_r，第三列是对应的 K_ij 的数值
K_sample = zeros(sample_size,3,40); % 40 是因为增益矩阵 K 应该是 4 行 10 列。
for i = 1:length
    for j = 1:length
        index = (i - 1)*length + j;
        l_l_ac = Leg_data_l(i,1); % 提取左腿对应的数据
        l_wl_ac = Leg_data_l(i,2);
        l_bl_ac = Leg_data_l(i,3);
        I_ll_ac = Leg_data_l(i,4);
        l_r_ac = Leg_data_r(j,1); % 提取右腿对应的数据
        l_wr_ac = Leg_data_r(j,2);
        l_br_ac = Leg_data_r(j,3);
        I_lr_ac = Leg_data_r(j,4);
        for k = 1:40
            K_sample(index,1,k) = l_l_ac;
            K_sample(index,2,k) = l_r_ac;
        end
    end
    K =
get_K_from_LQR(R_w,R_l,l_l,l_r,l_wl,l_wr,l_bl,l_br,l_c,m_w,m_l,m_b,I_w,I_ll
,I_lr,I_b,I_z,g, ...
    R_w_ac,R_l_ac,l_l_ac,l_r_ac,l_wl_ac,l_wr_ac,l_bl_ac,l_br_ac, ...

l_c_ac,m_w_ac,m_l_ac,m_b_ac,I_w_ac,I_ll_ac,I_lr_ac,I_b_ac,I_z_ac,g_ac, ...
    A,B,lqr_Q,lqr_R);
    % 根据指定的 l_l,l_r 输入对应的 K_ij 的数值
```

```

        for l = 1:4
            for m = 1:10
                K_sample(index,3,(l - 1)*10 + m) = K(l,m);
            end
        end
    end
end

% 创建收集全部 K_ij 的多项式拟合的全部系数的集合
KFit_Coefficients = zeros(40,6);
for n = 1:40
    K_Surface_Fit =
fit([K_sample(:,1,n),K_sample(:,2,n)],K_sample(:,3,n),'poly22');
    K_Fit_Coefficients(n,:) = coeffvalues(K_Surface_Fit); % 拟合并提取出拟合的
系数结果
end
Polynomial_expression = formula(K_Surface_Fit)
K_Fit_Coefficients =
sprintf([strjoin(repmat({'%.5g'},1,size(K_Fit_Coefficients,2))',' ') '\n'],
K_Fit_Coefficients.')
toc
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%得出控制矩阵 K 使用的函数%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function K =
get_K_from_LQR(R_w,R_l,l_l,l_r,l_wl,l_wr,l_bl,l_br,l_c,m_w,m_l,m_b,I_w,I_ll
,I_lr,I_b,I_z,g, ...
    R_w_ac,R_l_ac,l_l_ac,l_r_ac,l_wl_ac,l_wr_ac,l_bl_ac,l_br_ac, ...

l_c_ac,m_w_ac,m_l_ac,m_b_ac,I_w_ac,I_ll_ac,I_lr_ac,I_b_ac,I_z_ac,g_ac, ...
    A,B,lqr_Q,lqr_R)
% 基于机体以及物理参数，获得控制矩阵 A, B 的全部数值
A_ac = subs(A,[R_w R_l l_l l_r l_wl l_wr l_bl l_br l_c m_w m_l m_b I_w I_ll
I_lr I_b I_z g], ...
    [R_w_ac R_l_ac l_l_ac l_r_ac l_wl_ac l_wr_ac l_bl_ac l_br_ac l_c_ac ...
    m_w_ac m_l_ac m_b_ac I_w_ac I_ll_ac I_lr_ac I_b_ac I_z_ac g_ac]);
B_ac = subs(B,[R_w R_l l_l l_r l_wl l_wr l_bl l_br l_c m_w m_l m_b I_w I_ll
I_lr I_b I_z g], ...
    [R_w_ac R_l_ac l_l_ac l_r_ac l_wl_ac l_wr_ac l_bl_ac l_br_ac l_c_ac ...
    m_w_ac m_l_ac m_b_ac I_w_ac I_ll_ac I_lr_ac I_b_ac I_z_ac g_ac]);

% 根据以上信息和提供的矩阵 Q 和 R 求解 Riccati 方程，获得增益矩阵 K
% P 为 Riccati 方程的解，矩阵 L 可以无视
[P,K,L_k] = icare(A_ac,B_ac,lqr_Q,lqr_R,[],[],[]);
end

```

2.1.8 腿部控制器

腿部控制器的具体控制方法详见论文控制框图 16 即可，在论文第二节中的综合运动控制部分均有提及，包括了离地检测部分的实现。

3. 观测器设计

在轮腿机器人的控制系统中，打滑是一个很头疼的问题，机器人在非结构化场地运动时，难免会压倒小碎块、柱状或球状物导致打滑，而机器人的一个重要的状态量就是里程计，也就是机器人位置和速度观测，打滑会导致里程计严重不准（本身就不大准了）最终导致机器人姿态发散最后失衡倒下。那么这个时候我们实际上可以借鉴 LQG 算法的思想，将打滑作为较严重的噪声数据进行滤除，我们知道卡尔曼滤波器采用递归的方式进行状态估计，即通过先前的状态估计和最新的观测数据来更新当前的状态估计。这种递归更新的方式使得卡尔曼滤波器计算效率高，并且可以实时应用于动态系统。且卡尔曼滤波器基于贝叶斯推断原理，利用观测数据和系统模型进行最优状态估计。它通过融合过去的状态估计和最新的观测数据，提供具有最小均方误差的状态估计结果。这使得卡尔曼滤波器在噪声存在的情况下能够提供较为准确的状态估计。

假设机器人是一个质点，在走动的过程中，极小段时间内满足基本的匀速直线运动。根据运动学规律可以构建一个基本的运动学模型，得到运动方程：

$$x = vt + \frac{1}{2}at^2$$

$$v = v_0 + at$$

将加速度 a 看作控制量，加速度 a 可以通过 IMU 的加速度计直接测量得到，该模型为线性系统，得到状态转移方程为：

$$\begin{bmatrix} x_k \\ v_k \end{bmatrix} = \begin{bmatrix} 1 & dt \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_{k-1} \\ v_{k-1} \end{bmatrix} + \begin{bmatrix} 0.5dt^2 \\ 1 \end{bmatrix} a_k$$

观测方程理论上可以观测里程和集体速度，但是轮部的里程也是靠速度积分得到，在整个观测部分只选用了轮部的速度观测，观测方程如下：

$$z_k = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x_k \\ v_k \end{bmatrix}$$

将整个模型带入到 kalman 中即可：

$$\begin{aligned} \hat{x}_{k|k-1} &= F_{k,k-1} * x_{k-1} \\ \hat{P}_{k|k-1} &= F_{k,k-1} * P_{k-1} * F_{k,k-1}^T + Q_{k-1} \\ K_k &= \hat{P}_{k|k-1} * H_k^T [H_k * \hat{P}_{k|k-1} * H_k^T + R_k]^{-1} \\ x_k &= \hat{x}_{k|k-1} + K_k [z_k - H_k * \hat{x}_{k|k-1}] \\ P_k &= [I - K_k * H_k] * \hat{P}_{k|k-1} \end{aligned}$$

在 matlab 中加入白噪声，构建一个简单的匀加速直线运动模型。在整个模型中加入一个冲激信号来模拟压小碎石的状态，可以看到速度很快收敛，同时数据处理也很平滑，这里就不贴代码了，模型比较简单，仿真结果如下图 2：

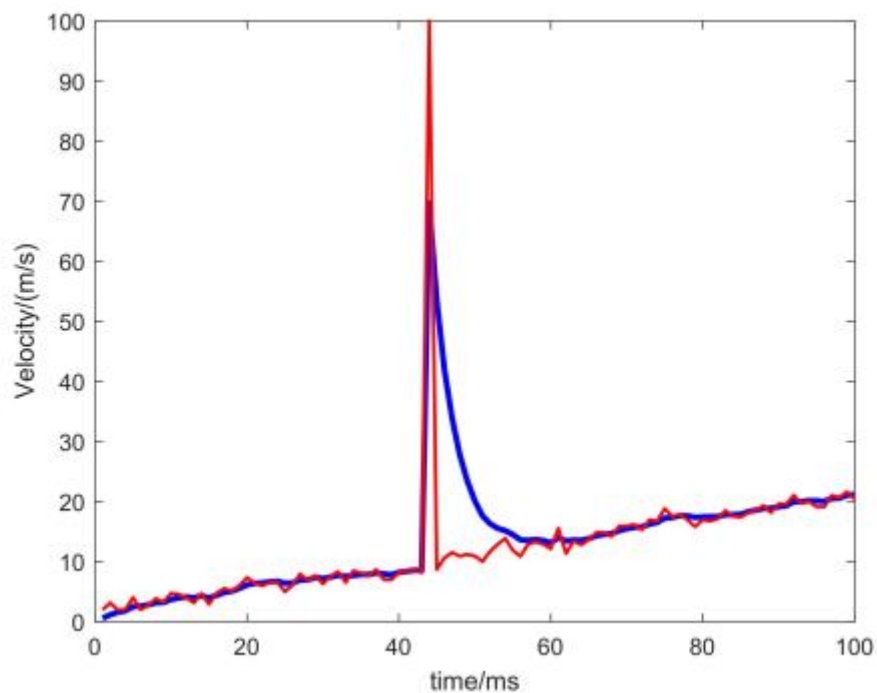


图2 卡尔曼滤波仿真结果

对于这个情况上交团队则是采用了类似 MPC 的思想，利用状态空间方程进行了 5~10 步的一个预测，根据预测值来修正轮子的扭矩，对于其他突发情况例如被冲撞，可能会有更好的效果，但算力消耗较大。

4. 仿真与结论

4.1 仿真

利用 Webots 软件对搭建如下图 3 的仿真场景与轮腿机器人：

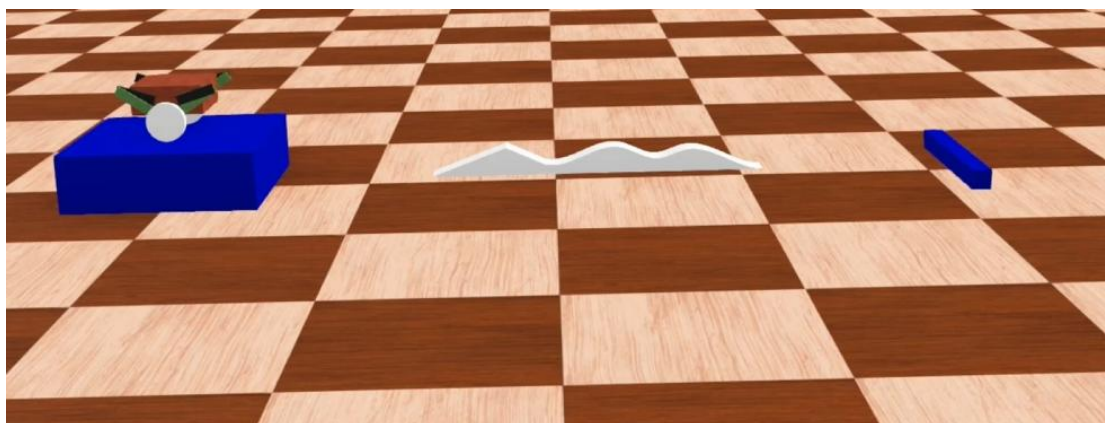


图3 Webots 仿真

4.2 程序代码

Python 代码：

```
import math
import numpy as np
import lagrange
```

```
import leg
import mymath
from controller import Motor, PositionSensor, Gyro, Accelerometer, Robot
# 初始化机器人和时间步长
robot = Robot()
timestep = int(robot.getBasicTimeStep())
# 初始化电机
motor_5 = Motor('motor5')
motor_6 = Motor('motor6')
motor_1 = Motor('motor1')
motor_3 = Motor('motor3')
motor_2 = Motor('motor2')
motor_4 = Motor('motor4')
# 初始化 IMU 等各种传感器
ps_5 = PositionSensor('ps5')
ps_6 = PositionSensor('ps6')
ps_1 = PositionSensor('ps1')
ps_3 = PositionSensor('ps3')
ps_2 = PositionSensor('ps2')
ps_4 = PositionSensor('ps4')
gyro = Gyro('gyro')
accelerometer = Accelerometer('accelerometer')
# 初始化电机编码器反馈
for motor in [motor_1, motor_2, motor_3, motor_4, motor_5, motor_6]:
    motor.setPosition(float('inf'))
    motor.setVelocity(0)
for sensor in [ps_1, ps_2, ps_3, ps_4, ps_5, ps_6, gyro, accelerometer]:
    sensor.enable(timestep)
# 初始化差分计算
d_t = timestep / 1000
Theta_b = mymath.Discreteness(d_t)
Fai = mymath.Discreteness(d_t)
Theta_wl = mymath.Discreteness(d_t)
Theta_wr = mymath.Discreteness(d_t)
Roll = mymath.Discreteness(d_t)
theta_l1 = mymath.Discreteness(d_t)
theta_l4 = mymath.Discreteness(d_t)
theta_r1 = mymath.Discreteness(d_t)
theta_r4 = mymath.Discreteness(d_t)
d_Ll = mymath.Discreteness(d_t)
d_Lr = mymath.Discreteness(d_t)
Ll = mymath.Discreteness(d_t)
# 设置初始差分
Ll.last_diff = 0.10456422824432195
```

```
theta_l1.last_diff = 3.447025
theta_r1.last_diff = 3.447025
theta_l4.last_diff = -0.305426
theta_r4.last_diff = -0.305426
# 初始化目标值和 PID 控制器
target_L0_l = 0.2
target_L0_r = 0.2
target_roll = 0
F0_control_l = mymath.PID_control(400, 2, 8000, target_L0_l)
F0_control_r = mymath.PID_control(400, 2, 8000, target_L0_r)
Froll_control = mymath.PID_control(3000, 1, 100, target_roll)
r = 0.10
current_time = 0
# 初始化离地检测标志位
Offground_l = 0
Offground_r = 0
# 初始化卡尔曼滤波器参数
x_k = np.matrix([[0], [0]])
x_hat_k = np.matrix([[0], [0]])
Q_c = np.matrix([[1 / 4 * d_t ** 4, d_t ** 3 / 2], [d_t ** 3 / 2, d_t ** 2]])
R_c = np.matrix([[1, 0], [0, 1]])
P = np.matrix([[1, 0], [0, 1]])
H_m = np.matrix([[1, 0], [0, 1]])
A = np.matrix([[1, d_t], [0, 1]])
B = np.matrix([[0], [0]])
# 主循环
while robot.step(timestep) != -1:
    current_time += d_t
    # 获取机器人状态
    dot_theta_b = gyro.getValues()[0]
    theta_b = Theta_b.Sum(dot_theta_b)
    dot_roll = gyro.getValues()[1]
    roll = Roll.Sum(dot_roll)
    dot_fai = gyro.getValues()[2]
    fai = Fai.Sum(dot_fai)
    acc_x = accelerometer.getValues()[0]
    acc_y = -accelerometer.getValues()[1]
    acc_z = accelerometer.getValues()[2]
    # 计算腿部角度和长度
    phi_l1 = 3.447025 - ps_1.getValue()
    phi_l4 = -0.305426 - ps_3.getValue()
    phi_r1 = 3.447025 - ps_2.getValue()
    phi_r4 = -0.305426 - ps_4.getValue()
```

```

    phi_l2, phi_l3, L0_l, phi0_l = leg.getPhi(phi_l1, phi_l4, 0.2, 0.3, 0.3,
0.2)
    phi_r2, phi_r3, L0_r, phi0_r = leg.getPhi(phi_r1, phi_r4, 0.2, 0.3, 0.3,
0.2)
    # 五连杆转化为单杆数据
    theta_wl = ps_5.getValue()
    theta_wr = ps_6.getValue()
    dot_theta_wl = Theta_wl.Diff(theta_wl)
    dot_theta_wr = Theta_wr.Diff(theta_wr)
    omega1l = theta_l1.Diff(phi_l1)
    omega4l = theta_l4.Diff(phi_l4)
    omega1r = theta_r1.Diff(phi_r1)
    omega4r = theta_r4.Diff(phi_r4)
    L0_speedl, test2 = leg.spd(omega1l, omega4l, 0.2, 0.3, 0.3, 0.2, 0.12, phi_l1,
phi_l4)
    L0_speedr, test4 = leg.spd(omega1r, omega4r, 0.2, 0.3, 0.3, 0.2, 0.12, phi_r1,
phi_r4)
    L0_spdl = L1.Diff(L0_l)
    L0_accl = d_L1.Diff(L0_speedl)
    L0_accr = d_Lr.Diff(L0_speedr)
    # 得到 LQR 全状态反馈的所有反馈参量
    s = r * (theta_wl + theta_wr) / 2
    dot_s_b = r * (dot_theta_wl + dot_theta_wr) / 2
    dot_s = dot_s_b + 0.5 * (L0_l * test2 * math.cos(theta_b) + L0_r * test4
* math.cos(theta_b)) + 0.5 * (L0_speedl * math.sin(theta_b) + L0_speedr *
math.sin(theta_b))
    x_k = np.matrix([[dot_s], [acc_y]])
    x_k = A * x_k
    z = H_m * x_k
    x_hat_k, x_hat_minus, P = mymath.LinearKalman(A, B, Q_c, R_c, H_m, z, x_hat_k,
P)
    dot_s_k = x_hat_k[0, 0]
    # 构建状态向量
    real_state = np.matrix([[s],
                             [dot_s_k],
                             [fai],
                             [dot_fai],
                             [theta_b],
                             [dot_theta_b],
                             [phi0_l],
                             [test2],
                             [phi0_r],
                             [test4]])
    # 定义期望状态

```

```

if 0 < current_time <= 1:
    expect_state = np.matrix([[0], [0], [0], [0], [0], [0], [0], [0], [0],
[0]])
elif 1 < current_time <= 6:
    expect_state = np.matrix([[1 * (current_time - 1)], [1], [0], [0], [0],
[0], [0], [0], [0], [0]])
elif 6 < current_time <= 8:
    expect_state = np.matrix([[1 * (current_time - 1)], [1], [-1.57 *
(current_time - 6)], [-1.57], [0], [0], [0], [0], [0], [0]])
elif 8 < current_time <= 11:
    expect_state = np.matrix([[1 * (current_time - 1)], [1], [-3.14], [0],
[0], [0], [0], [0], [0], [0]])
elif 11 < current_time <= 18:
    expect_state = np.matrix([[10], [0], [-3.14], [0], [0], [0], [0], [0],
[0], [0]])
else:
    break
# 调整 PID 目标值
if 12 < current_time <= 14:
    F0_control_l.targ_value = 0.205
    F0_control_r.targ_value = 0.205
if 14 < current_time <= 16:
    F0_control_l.targ_value = 0.325
    F0_control_r.targ_value = 0.325
if 16 < current_time <= 18:
    F0_control_l.targ_value = 0.205
    F0_control_r.targ_value = 0.203
# 计算控制力矩
K = lagrange.K(target_L0_l, target_L0_r)
if Offground_l:
    K[0] = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    K[2] = [0, 0, 0, 0, K[2, 4], K[2, 5], 0, 0, 0, 0]
if Offground_r:
    K[1] = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    K[3] = [0, 0, 0, 0, 0, 0, K[3, 6], K[3, 7], 0, 0]
U = K * (expect_state - real_state)
T_l = float(U[0][0])
T_r = float(U[1][0])
T_pl = float(U[2][0])
T_pr = float(U[3][0])
# 计算关节力矩
JRM_L = leg.Mat_JRM(phi0_l, phi_l1, phi_l2, phi_l3, phi_l4, L0_l, 0.2, 0.2)
JRM_R = leg.Mat_JRM(phi0_r, phi_r1, phi_r2, phi_r3, phi_r4, L0_r, 0.2, 0.2)
dF_0_l = -F0_control_l.position_pid(L0_l)

```

```

dF_0_r = -F0_control_r.position_pid(L0_r)
dF_roll = -Froll_control.position_pid(roll)
# 计算腿部力和力矩
F_bl = -dF_roll + dF_0_l - 30
F_br = dF_roll + dF_0_r - 30
T_JOINT_l = JRM_L * np.matrix([[F_bl], [T_pl]])
T_JOINT_R = JRM_R * np.matrix([[F_br], [T_pr]])
T1_l = float(T_JOINT_l[0][0])
T1_r = float(T_JOINT_l[1][0])
T2_l = float(T_JOINT_R[0][0])
T2_r = float(T_JOINT_R[1][0])
# 离地检测判断
acc_zwl = (acc_z - 9.8) - L0_accl * math.cos(theta_b)
acc_zwr = (acc_z - 9.8) - L0_accr * math.cos(theta_b)
Pl = F_bl * math.cos(theta_b) + T_pl * math.sin(theta_b) / L0_l
Pr = F_br * math.cos(theta_b) + T_pr * math.sin(theta_b) / L0_r
F_nl = Pl - 9.8 - acc_zwl
F_nr = Pr - 9.8 - acc_zwr
if F_nl > -7:
    Offground_l = 1
else:
    Offground_l = 0
if F_nr > -7:
    Offground_r = 1
else:
    Offground_r = 0
# 设置电机扭矩
if 0 < current_time <= 10 * d_t:
    motor_5.setTorque(T_l)
    motor_6.setTorque(T_r)
elif 5.10 < current_time <= 5.20:
    motor_1.setTorque(15)
    motor_3.setTorque(-15)
    motor_2.setTorque(15)
    motor_4.setTorque(-15)
    motor_5.setTorque(T_l)
    motor_6.setTorque(T_r)
elif 5.20 < current_time <= 5.30:
    motor_1.setTorque(-10)
    motor_3.setTorque(10)
    motor_2.setTorque(-10)
    motor_4.setTorque(10)
    motor_5.setTorque(T_l)
    motor_6.setTorque(T_r)

```

```
else:
    motor_1.setTorque(T1_l)
    motor_3.setTorque(T1_r)
    motor_2.setTorque(T2_l)
    motor_4.setTorque(T2_r)
    motor_5.setTorque(T_l)
    motor_6.setTorque(T_r)
```

4.3 18s 仿真结果与分析

整个轮腿机器人 18s 分别进行以下几个阶段，抬腿->起步->下台阶->过单边桥->跳跃障碍->折返->伸腿，运动流程见下图 4。

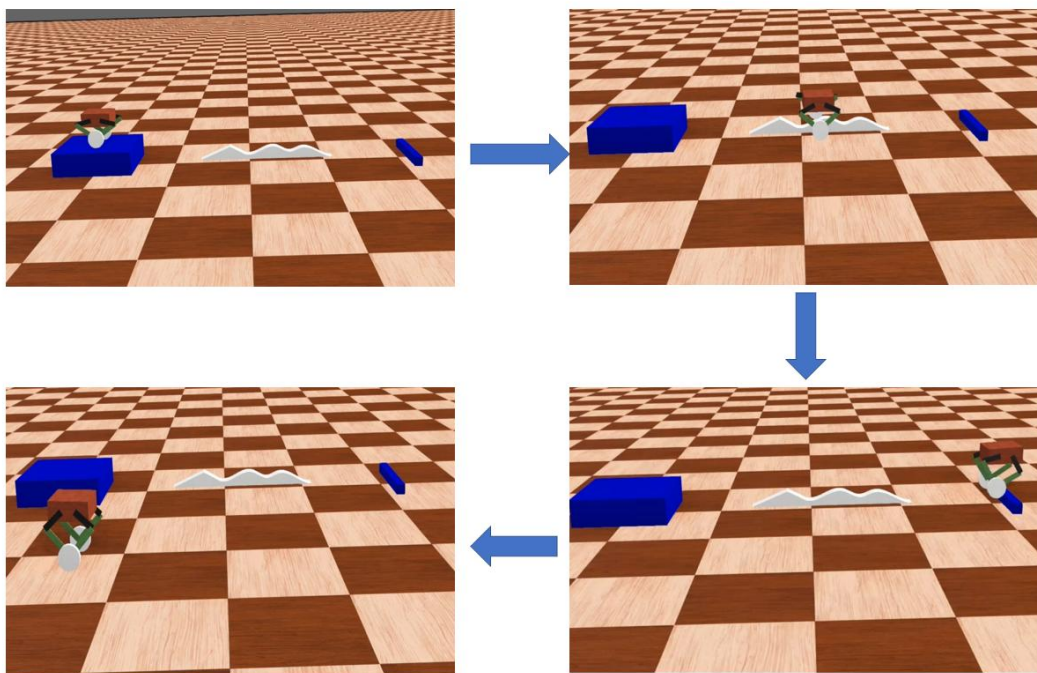


图 4 运动流程图

左右腿长变化的数据如下图 5，由图中可以看出由于腿长环的存在，两腿的期望长度相同在大部分时候腿长都是相同的，但在 2~4s 时腿长却变化方向相反，这是由 roll 环的存在为了保证机体的 roll 轴水平在通过单边桥时会强制更改两腿期望腿长。跳跃动态就是在短时间内迅速更改期望腿长完成。

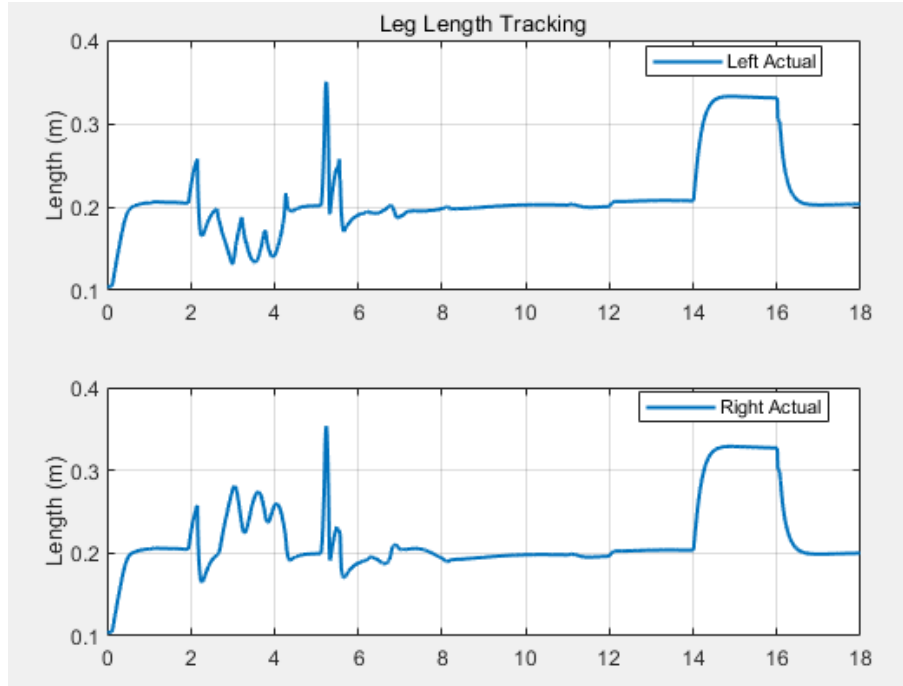


图 5 腿长变化

腿部摆动角度以及机体 pitch 角与轮子里程计共同组成大平衡环，由前面设计的 Q 矩阵可知对于机体倾角的惩罚值远大于其他几项，期望状态为平衡点，在正常运动时会优先通过摆动腿和移动位置来保持机体 pitch 角的稳定，如下图 6，图 7 为腿部摆角与机体倾角数据。

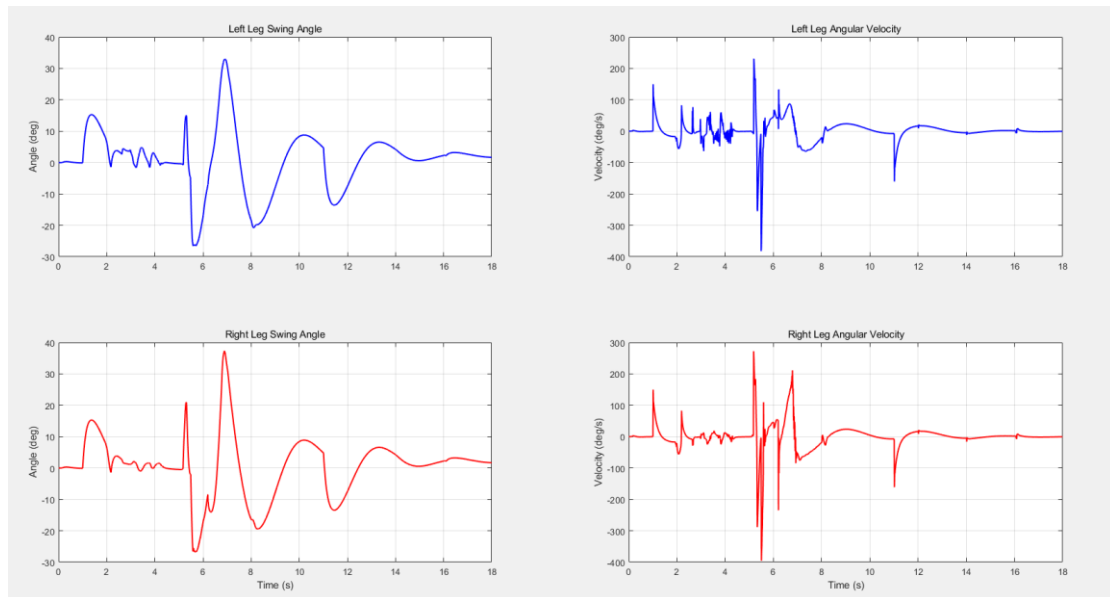


图 6 腿部摆角

仿真结果表明，采用 LQR 算法结合虚拟模型控制（ VMC ）的控制策略能够有效实现机器人的平衡控制和纵向运动控制。在加速和减速过程中，机器人能够通过调整腿部姿态和驱动轮力矩，保持上层机构的稳定性。仿真中施加的外力扰动测试验证了控制系统的鲁棒性，机器人能够在受到干扰后迅速恢复稳定。通过 PD 控制实现转向控制，并通过 PID 控制实现腿长调节和横滚角补偿，机器人能够在复杂地形下保持机身水平。在离地情况下，通过调整反馈增益矩阵，机器人能够保持空中姿态稳定，并实现平稳落地。以后会针对实际应用中可能出现的复杂情况，进一步优化控制算法，提高系统的鲁棒性和适应性。

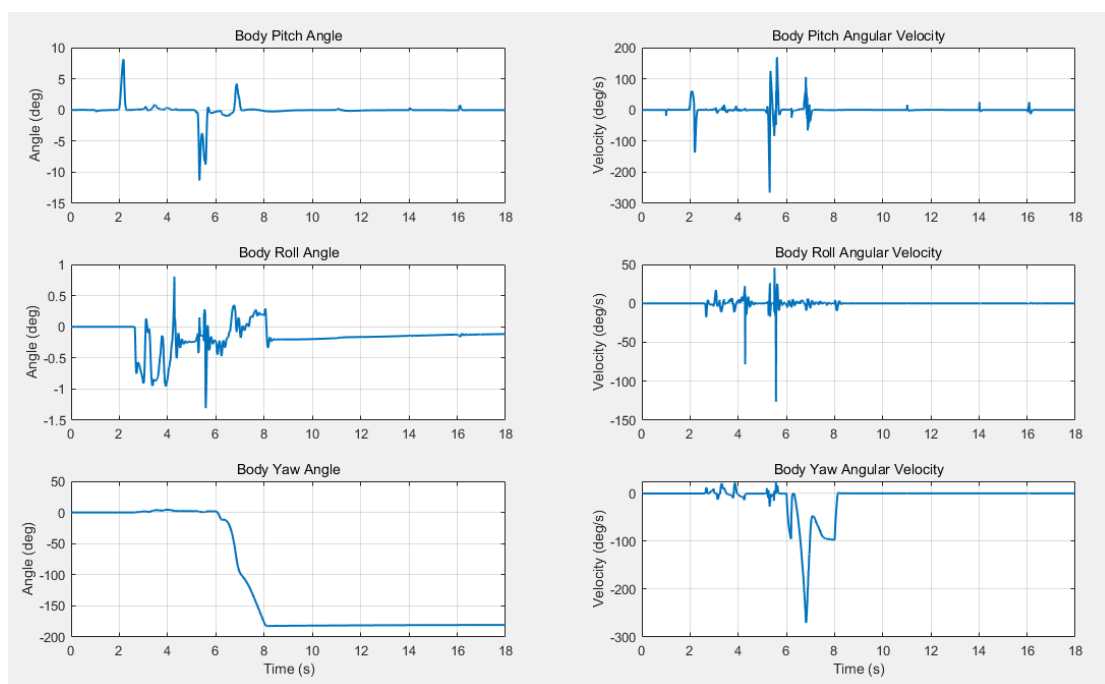


图 7 机体三轴角度

R 矩阵也同样有较大的作用，如在 robomaster 比赛中腿部电机不计入底盘功率，轮电机会计入功率，R 的适当取值就可以决定机器人腿处理大还是轮出力大。

篇幅有限并未进行不同 Q R 矩阵的对比。

参考文献

- [1] BJELONIC M, BELLICOSO C D, DE VIRAGH Y, et al. Keep rollin—whole-body motion control and planning for wheeled quadrupedal robots [J]. IEEE Robotics and Automation Letters, 2019, 4(2): 2116-2123.作者.文章题名 [J].期刊名, 年, 卷(期): 起止页码.
- [2] KLEMM V, MORRA A, SALZMANN C, et al. Ascento: A two-wheeled jumping robot [C]. International Conference on Robotics and Automation. Piscataway, USA: IEEE, 2019: 7515-7521.
- [3] KLEMM V, MORRA A, GULICH L, et al. LQR-assisted whole-body control of a wheeled bipedal robot with kinematic loops [J]. IEEE Robotics and Automation Letters, 2020, 5(2): 3745-3752.
- [4] BELLICOSO C D, GEHRING C, HWANGBO J, et al. Perception-less terrain adaptation through whole body control and hierarchical optimization [C]. IEEE-RAS 16th International Conference on Humanoid Robots. Piscataway, USA: IEEE, 2016: 558-564.
- [5] 谢惠祥, 罗自荣, 尚建忠. 四足机器人对角小跑动态控制 [J]. 国防科技大学学报, 2014, 36(4): 146-151.
- [6] ZHANG K, WANG J Z, PENG H, et al. Model predictive control based path following for a wheel-legged robot [C]. 32nd Chinese Control and Decision Conference. Piscataway, USA: IEEE, 2020: 3925-3930.
- [7] WANG S, LU H, HOU F. An improved computing model for a two-wheeled self-balancing vehicle's state determination [C]. 8th International Congress on

- Image and Signal Processing. Piscataway, USA: IEEE, 2015: 1379-1384.
- [8] SHEN Y, CHEN G, LI Z, et al. Cooperative control strategy of wheel-legged robot based on attitude balance [J] . Robotica, 2022, 41(2): 566-586.
- [9] 高建, 葛文杰. 单关节跳跃机器人模糊自适应轨迹跟踪控制 [J] . 计算机仿真, 2013, 30(2): 331-335.
- [10] 韩连强, 陈学超, 余张国, 等. 面向离散地形的欠驱动双足机器人平衡控制方法 [J] . 自动化学报, 2022, 48(9): 2164-2174.
- [11] Boston Dynamics. Handle [EB/OL] . (2018-06-06)[2022-02-16]. <https://www.bostondynamics.com/handle>.
- [12] LEE J, BJELONIC M, HUTTER M. Control of wheeled-legged quadrupeds using deep reinforcement learning [C] . 25th International Conference on Climbing and Walking Robots and Mobile Machine Support Technologies. Piscataway, USA: IEEE, 2022: 119-127.